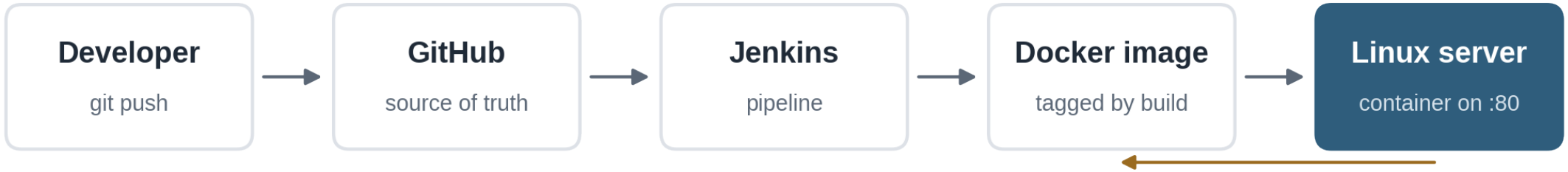
Release Console

Automated CI/CD: from a GitHub commit to a verified deployment



The problem

- Manual releases are slow, inconsistent and hard to undo.
- Nobody can tell which commit is running on the server.

The solution

- Every push is tested, packaged and deployed by Jenkins.
- A bad release is rejected or rolled back automatically.

Tools used

- Git and GitHub for source control.
- Docker for repeatable builds and runtime.
- Jenkins on Ubuntu 24.04, local VM or AWS EC2.

8 pipeline stages

10 automated tests

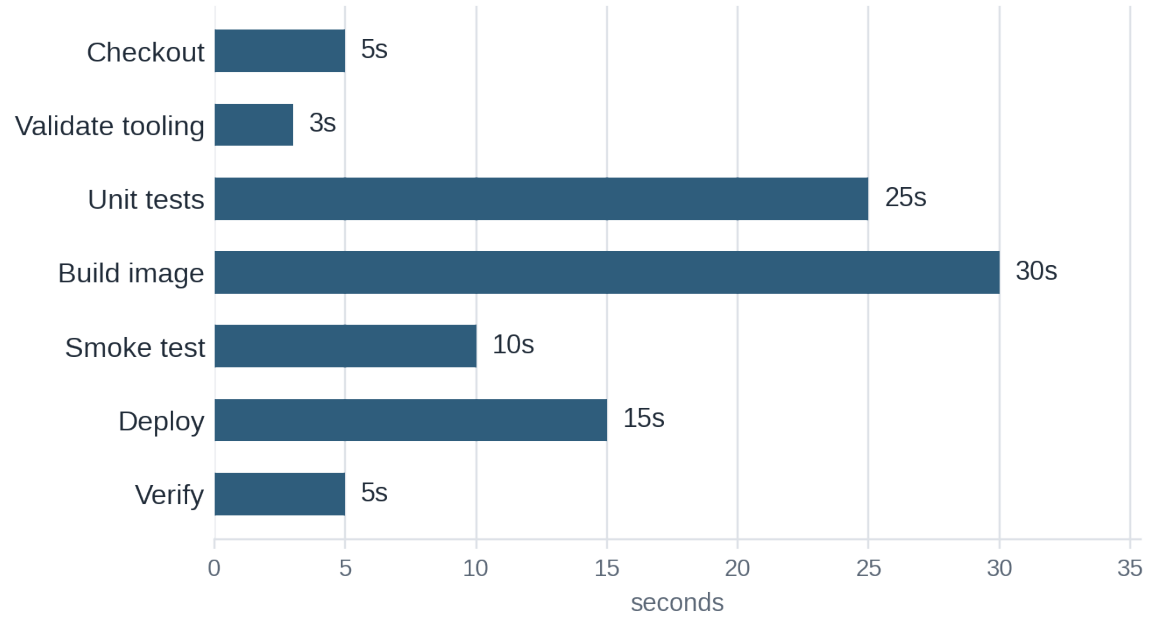
3 safety nets

0 manual steps after git push

From commit to live in seven steps

About 93 seconds end to end (indicative), with three safety nets along the way

Time spent in each stage (indicative)



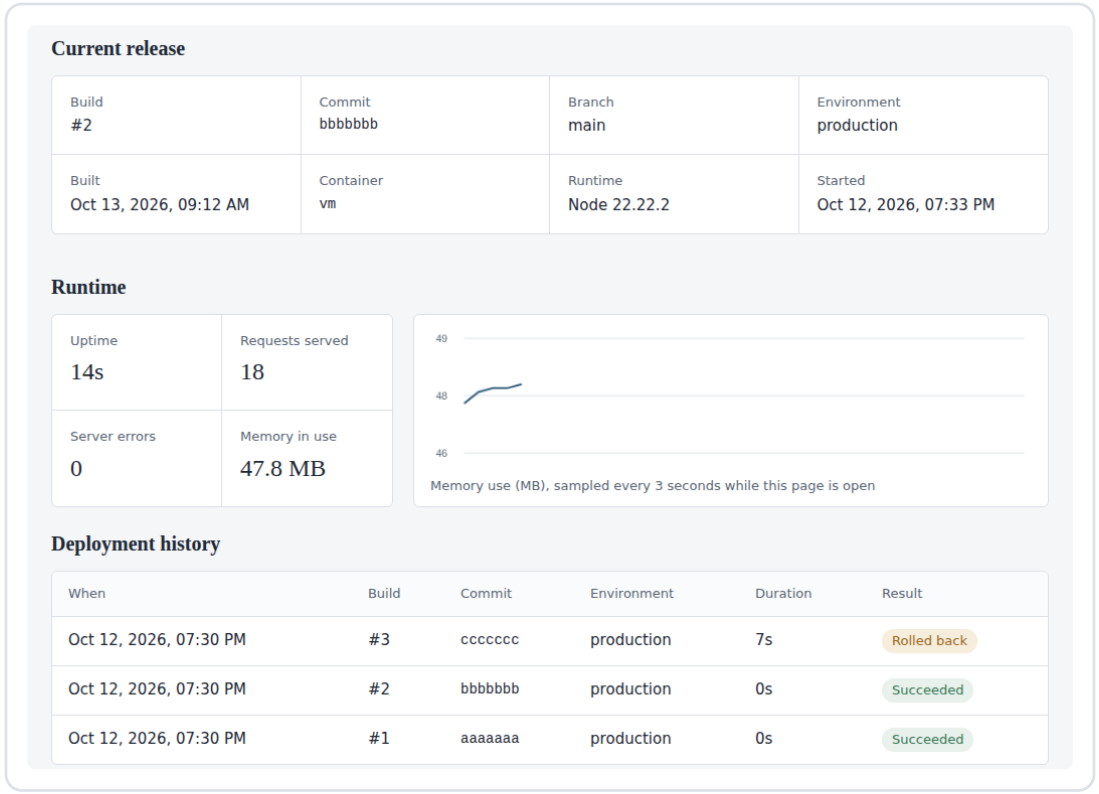
Safety nets

- **Unit tests gate the build**
  Lint and 10 tests run inside the Docker build. A failure stops the pipeline before an image exists.
- **Smoke test before deploy**
  The new image starts on a throwaway port and must answer its health check and report its own build number.
- **Health check and rollback**
  The old container is kept until the new one is healthy. If not, it is restored and the attempt is logged.

Finally, the live page is checked to confirm it serves the build just deployed.

What was delivered

A working pipeline, and a page that proves each deployment



The deployed Release Console, running locally: build, health and deployment history.

Deliverables

- GitHub repository with branching and commit conventions
- Dockerfile: multi-stage, non-root, health-checked
- Jenkinsfile: 8-stage declarative pipeline
- Deployment scripts with automatic rollback
- Deployed web application with live release history
- Workflow, setup and troubleshooting documentation

Skills developed

Git · GitHub · Linux · Docker · Jenkins · CI/CD · AWS EC2 · Troubleshooting